Rapport de Modélisation Orientée Objet

Application ZooKeeper

Équipe Projet POO

23 octobre 2026

1 Introduction

Ce document formalise les choix d'architecture logicielle retenus pour la gestion du parc zoologique **ZooKeeper**. Le modèle de données a été optimisé afin d'assurer une modélisation fidèle du domaine tout en respectant les concepts fondamentaux de la programmation orientée objet en Java.

2 Décisions de Conception

2.1 1. Hiérarchie du Personnel (`Employe`)

Afin de respecter le principe de responsabilité unique (SRP), les rôles ont été spécialisés :

- **Classe Abstraite** `Employe` : Facteur commun regroupant le nom et le matricule.
- **Classe** `Soigneur` : Responsable de la logistique des enclos (`assignerEnclos`) et de l'alimentation (`planifierRepas`).
- **Classe** `Veterinaire` : Dédiée exclusivement aux actes de diagnostic et aux soins médicaux des animaux.

2.2 2. Modélisation des Relations : Composition vs Agrégation

- **Composition (`Zoo` possède `Enclos`)** : Les enclos font partie intégrante de l'infrastructure du parc. La suppression du zoo entraîne la destruction de ses enclos.
- **Agrégation (`Enclos` héberge `Animal`)** : Un animal loge dans un enclos, mais conserve une existence propre. Si un enclos est rénové ou supprimé, les animaux sont déplacés.

2.3 3. Interfaces Multiples (`Volant`, `Nageur`, `Soignable`)

Conformément à la consigne demandant d'illustrer la mise en œuvre de plusieurs interfaces par classe :

- Les classes `Oiseau` et `Mammifere` implémentent simultanément les interfaces `Volant` et `Nageur` (ex : canards, chauves-souris, cétacés).
- L'interface `Soignable` est implémentée au sommet de la hiérarchie dans `Animal`.

2.4 4. Attributs et Logique Conditionnelle

Les attributs `regime` et `statutSante` sont maintenus sous forme de chaînes de caractères (`String`). Les règles de compatibilité d'alimentation et de génération d'alertes de santé sont traitées directement par des structures conditionnelles (`equalsIgnoreCase`) au sein des classes métiers.